

Deep Learning Foundations

Chapter 10: Convolutional Neural Networks

Sashreek Kumar

10.3)

Solution:

Calculating each element of the output matrix Y by applying the kernel to the input patches:

$$\begin{aligned}Y_{1,1} &= (-2 \times 2) + (0 \times 5) + (4 \times 0) + (6 \times 6) = 32 \\Y_{1,2} &= (-2 \times 5) + (0 \times -3) + (4 \times 6) + (6 \times 0) = 14 \\Y_{1,3} &= (-2 \times -3) + (0 \times 0) + (4 \times 0) + (6 \times -4) = -18 \\Y_{2,1} &= (-2 \times 0) + (0 \times 6) + (4 \times -1) + (6 \times -3) = -22 \\Y_{2,2} &= (-2 \times 6) + (0 \times 0) + (4 \times -3) + (6 \times 0) = -24 \\Y_{2,3} &= (-2 \times 0) + (0 \times -4) + (4 \times 0) + (6 \times 2) = 12 \\Y_{3,1} &= (-2 \times -1) + (0 \times -3) + (4 \times 5) + (6 \times 0) = 22 \\Y_{3,2} &= (-2 \times -3) + (0 \times 0) + (4 \times 0) + (6 \times 0) = 6 \\Y_{3,3} &= (-2 \times 0) + (0 \times 2) + (4 \times 0) + (6 \times 3) = 18\end{aligned}$$

The resulting feature map is:

$$\begin{bmatrix} 32 & 14 & -18 \\ -22 & -24 & 12 \\ 22 & 6 & 18 \end{bmatrix}$$

10.6)

Solution:

To maintain the spatial dimensions of the input throughout a convolutional layer, we must determine the padding P such that the output dimension equals the input dimension. Given an input dimension J , kernel size M , stride S , and padding P , the output dimension is defined by:

$$\text{Output} = \frac{J + 2P - M}{S} + 1$$

To achieve **Same Padding** (where Output = J) under the assumption of a unit stride ($S = 1$), we set up the following identity:

$$\frac{J + 2P - M}{1} + 1 = J$$

Algebraic Derivation: Solving for P , we have:

$$\begin{aligned} J + 2P - M + 1 &= J \\ 2P - M + 1 &= 0 \\ 2P &= M - 1 \\ \implies P &= \frac{M - 1}{2} \end{aligned}$$

Conclusion: For a convolution with stride $S = 1$, applying a padding of $P = \frac{M-1}{2}$ ensures that the feature map preserves the input's spatial resolution. Note that this implies M should be an odd number to ensure an integer value for P , which is a primary reason why 3×3 or 5×5 kernels are standard in deep learning architectures.

10.7)

Solution:

Consider an input of size $J \times K$, a square filter of size $M \times M$, padding P , and stride S .

1. Padded Dimensions Applying padding P to all sides of the input increases the effective dimensions to:

$$(J + 2P) \times (K + 2P)$$

2. Spatial Constraints The kernel moves across the input in steps of size S . To find the number of valid positions, we calculate the total traversable distance $(J + 2P - M)$ and divide by the stride. Including the initial starting position, the dimensions of the resulting feature map are:

$$\left(\frac{J + 2P - M}{S} + 1 \right) \times \left(\frac{K + 2P - M}{S} + 1 \right)$$

3. Divisibility Constraint For the output to be well-defined without discarding information or requiring asymmetrical padding, the following condition must be met:

$$(J + 2P - M) \equiv 0 \pmod{S} \quad \text{and} \quad (K + 2P - M) \equiv 0 \pmod{S}$$

10.8)

Solution:

1. Mathematical Formulas The number of **Learnable Parameters** for a convolutional layer is determined by the kernel size, the number of input channels, and the number of filters (output channels), plus a bias term for each filter:

$$\text{Parameters} = (K \times C_{in} + 1) \times C_{out}$$

where K is the spatial area of the filter (e.g., $M \times M$).

The number of **Connections** represents the total number of arithmetic operations performed, which depends on the size of the resulting feature map (C_{layer}):

$$\text{Connections} = C_{layer} \times (K \times C_{in} + 1)$$

2. Layer-wise Breakdown The following table summarizes the computational requirements across the network:

Layer	Total Connections	Learnable Parameters
Convolution 1	96,337,920	1,792
Convolution 2	1,852,899,328	36,928
Convolution 3	926,449,664	73,856
Convolution 4	1,852,899,328	147,584
Convolution 5	926,449,664	295,168
Convolution 6	1,852,899,328	590,080
Convolution 7	1,852,899,328	590,080
Convolution 8	926,449,664	1,180,160
Convolution 9	1,852,899,328	2,359,808
Convolution 10	1,852,899,328	2,359,808
Convolution 11	462,522,368	2,359,808
Convolution 12	462,522,368	2,359,808
Convolution 13	462,522,368	2,359,808
FCN 1	102,764,544	102,764,544
FCN 2	16,781,312	16,781,312
FCN 3	4,097,000	4,097,000
Total	15,504,292,840	138,357,544

Observation: Notice that while the Convolutional layers account for the vast majority of the **Connections** (computational load), the Fully Connected Layers (FCN) dominate the total number of **Learnable Parameters** (memory footprint).